Artificial Intelligence Lab

Dibyasundar Das

27 July 2026

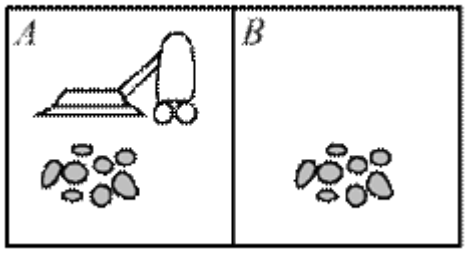
LAB 01

> **NB**
>
> All Coding must be done on python

Experiment 1:

Design a Vacuum Cleaner World Agent (2×1 Grid) using Python (Function-based, No Classes)

- Design a Vacuum Cleaner World of size **2×1** using matrix representation of cells.



- Place dirt randomly in the cells (each cell is either `0` = Clean or `1` = Dirty) — this represents the **true, hidden state** of the environment.
- Place the agent randomly in one of the cells.
- The agent maintains its **own knowledge** of the world, separate from the true hidden environment. Initially, the agent knows nothing about any cell (**including its own starting cell**) until it **perceives** it.
- The agent can move **Left or Right** (since the grid is 2×1) — but only when the **current cell is known to be clean**.
- If the current cell is **dirty**, the agent must **vacuum** the cell first before it can move.
- **The agent will never know if a cell is clean or dirty unless it has physically visited (perceived) that cell.** Unvisited cells must be treated as *unknown*, not assumed clean.

- The simulation should continue until the **agent's own knowledge** confirms all cells are clean, or a maximum number of iterations is reached — the stopping condition must **not** rely directly on the hidden true environment state, since that would let the agent "know" things it never sensed.
- The example output is as follows:

```
Initial Stage:
[A(),D][?]

Run 1:
[A(V),C][?]

Run 2:
[A(R),C][?]

Run 3:
[C][A(),D]

Run 4:
[C][A(V),C]

All cells are clean! Task Completed.
```

Notation explained

Symbol	Meaning
`D`	Dirty cell (known to agent)
`C`	Clean cell (known to agent)
`?`	Unknown cell — agent has not yet visited/perceived it
`V`	Agent just vacuumed this cell
`A()`	Agent present, no action taken this step (just perceiving/starting)
`A(R)` / `A(L)`	Agent present, decided to move Right / Left

You may use the following functions:

1. `create_environment()` – creates and returns the 2×1 array representing the **true hidden state** of the world (0 = Clean, 1 = Dirty).

2. `place_dirt(env)` – randomly places dirt (`1`) in cells of the grid.
3. `place_agent()` – randomly selects and returns a starting position (cell index) for the agent.
4. `create_knowledge()` – creates and returns the agent's knowledge array, where every cell starts as `None` (unknown).
5. `perceive(env, knowledge, pos)` – lets the agent sense the **true status of only the cell it currently occupies**, updating its own knowledge array.
6. `is_dirty(env, pos)` – checks whether the agent's current cell is actually dirty (used to decide the next action).
7. `vacuum(env, knowledge, pos)` – cleans the current cell in the true environment **and** updates the agent's knowledge to reflect it is now clean.
8. `get_possible_moves(pos)` – returns the list of valid moves (Left/Right) from the current position based on grid boundaries.
9. `move_agent(pos)` – moves the agent to a randomly chosen valid neighboring cell and returns the new position along with the direction moved.
10. `knowledge_all_clean(knowledge)` – checks whether the agent **believes** (based only on its own knowledge) that all cells are clean; returns `False` if any cell is still unknown (`None`) or known-dirty.
11. `all_clean(env)` – checks the **true** hidden environment state (used only for reference/debugging, not for the agent's stopping decision).
12. `display_environment(knowledge, agent_pos, action, step)` – prints the current state of the grid (from the agent's knowledge, with `?` for unvisited cells) and the agent's action at each step.
13. `run_vacuum_world()` – main driver function that ties everything together and runs the simulation loop until the agent's knowledge confirms all cells are clean, or the maximum step limit is reached.

Experiment 2

- Increase the grid to .